

1. Destructuring Assignment

👉 Object:

```
const person = {name: "Manish", age: 25};  
const {name, age} = person;
```

👉 Array:

```
const arr = [10, 20];  
const [a, b] = arr;
```

✦ *Memory*: “Extract values easily from object/array”

2. Simple Web Server (Node.js)

```
const http = require('http');  
  
http.createServer((req, res) => {  
  res.write("Hello World");  
  res.end();  
}).listen(3000);
```

✦ *Memory*: createServer → listen → response

3. Core Module

👉 Built-in modules in Node.js

Examples: http, fs, path

✦ *Memory*: “No install needed”

4. Middleware (ExpressJS)

👉 Function between request & response

```
app.use((req, res, next) => {  
  console.log("Middleware");  
  next();  
});
```

✦ *Memory*: “Request → Middleware → Response”

5. Event Loop (Node.js)

👉 Handles async tasks (non-blocking)

📌 *Memory*: “Runs tasks in background without blocking”

6. AngularJS

👉 JavaScript framework for frontend (SPA)

📌 *Memory*: “Frontend + dynamic pages”

7. Spread Syntax (...)

👉 Expands array/object

```
let arr2 = [...arr1];
```

📌 *Memory*: “Copy / Merge easily”

8. TypeScript Advantages

- Type safety
- Better error detection

📌 *Memory*: “JS + Types = Safe code”

9. package.json Attributes

- name
- version
- dependencies
- scripts

📌 *Memory*: “Project details + dependencies”

10. Types of Streams (Node.js)

- Readable

- Writable
- Duplex
- Transform

📌 *Memory:* “Read, Write, Both, Modify”

11. Why ExpressJS?

- Easy routing
- Middleware support
- Fast backend development

📌 *Memory:* “Simplifies Node.js”

12. Arrow Functions (ES6)

```
const add = (a,b) => a+b;
```

👉 No this, shorter syntax

📌 *Memory:* “Short + no own this”

13. MEAN Stack

- MongoDB
- ExpressJS
- Angular
- Node.js

📌 *Memory:* “Full JS stack”

14. AngularJS ng-repeat (Table)

```
<tr ng-repeat="s in students">  
  <td>{{s.name}}</td>  
</tr>
```

📌 *Memory:* “Loop in HTML”

15. Services (AngularJS)

👉 Reusable logic

Example: \$http

```
$http.get("api").then(...)
```

📌 *Memory:* “Common functions for app”